LUND UNIVERSITY

Real-Time Systems (FRTN01)

Project, VT2026

Group 1: MinSeg

Adam Holmqvist & Axel Lundquist & Axel Sjaunja & Mattias O'Hearn

***Supervisor**: Yde Sinnema*

Introduction

This project aims to develop the control and communication system for the MinSeg self-balancing robot. The main objective is to implement an LQG controller with Kalman filtering for state estimation. In practice, the system will be built using modular software architecture in Arduino. Initially, all computation and communication will be run locally on the arduino microcontroller, with the possibility of later extending to wireless Bluetooth communication.

Program structure

To control MinSeg an Arduino Mega 2560 will be used, which means the code will be written in Arduino C++ using the Arduino IDE. The setup() function will contain initialization of sensors, control parameters and calibration variables. The main loop() function will contain the overall program logic, including the periodic control loop that must execute at a fixed sampling rate.

or the other way around?

Inside of this periodic loop the program will read the sensors, update the states estimates and then calculate the control output. Since it's a fast system the controller will need to be kept simple and deterministic so that it meets its deadlines. This also means that functions that are not equally time sensitive, for example data plotting or changing the parameters will happen at a lower priority, placed outside of the main control loop. good idea

In order to reduce development time libraries will be used to communicate with the low level hardware. Communication between the arduino and the IMU will be with I2C, so a library for that and the MPU6050 is needed. RASPLib will not be used in the final implementation, however it may be used during early development as a stepping stone and as a reference. So this can reduce the development time in the initial phase while the final controller is still written directly in C++.

On top of these libraries, helper classes and methods will be developed to clearly structure the code. These could include IMU reading and calibration, state estimation and modes such as startup, calibration, balancing and stop. If bluetooth is added, methods for sending and receiving data are needed as well as a simple user interface.

Will you have different threads / shared variables?

Operator communication

The operator must be able to start and stop the controller, calibrate the sensors, monitor signals in real time and adjust controller parameters when needed. One project requirement is specifically that data should be received and plotted in real time and that simple commands or parameter changes should be sent to the MinSeg during operation.

At the start operator communication will most likely be done through USB since that is the simplest setup for development, programming, and debugging. In a later stage the aim is to

move toward Bluetooth-based wireless communication since the MinSeg hardware supports an HC-06 Bluetooth module.

The operator interface will thus serve as both a command interface and a monitoring tool. A later goal is making the interface support remote operation of the MinSeg by making a simple user interface with buttons for steering.

Control principles

what are the states of the system?
what do you measure? what do you actuate?

The first control method will be LQG. This means using Kalman filtering to estimate the system states from the noisy IMU and encoder measurements together with LQ state feedback to stabilize the upright equilibrium. This feels like a suitable starting point because the MinSeg is an unstable balancing robot and the previous laboration from the advanced control course already provides a starting point for estimation and control framework for this type of system.

For sensor fusion the project description highlights that the gyroscope is useful at high frequencies but suffers from drift, while the accelerometer is useful at low frequencies but is noisy. This motivates a Kalman filter. Since we want to start from LQG, Kalman filtering is the natural choice. If time permits alternative methods such as PID or nonlinear control can be tested and compared against the LQG baseline. The project text also explicitly recommends developing a mathematical model in parallel since this enables methods such as pole placement and LQG.

The simulink block diagram below is from the advanced control theory course. This will be used as a guide and inspiration for solving the software portion of the project.

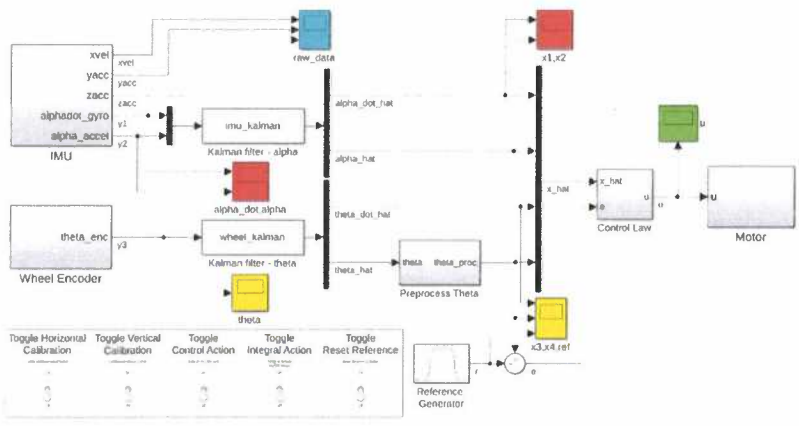

Figure 1: Simulink block diagram of the MinSeg controller

Project plan

The project is divided into a main part (see "Main plan" below) and further improvements (see "Further improvements").

The initial idea is to follow the main plan. It includes the expectations that the group has personally decided as a bare minimum for the project to be successful. More specifically, the main plan consists of: *Specifications* which consists of initial decisions that will be used for the remaining portion of the project (if not changed during the *Further improvements*); *Procedure* that is divided into various phases (from 1 to 9) of the project written in sequential order, where lower phases are necessary for completion before moving onto the next phase; and *Timeframe* that consists of a table explaining the timeframe for each phase. When overlapping phases occur according to the *Timeframe*, the group will divide into groups of 2 where each subgroup focuses on a respective phase that allows one phase to become finished and a new one introduced for a more smooth transition. The project report will be written as soon as measurement data can be taken (week 17) and include observations from there on. If time allows, further improvements will be implemented, and the amount of further improvements depends on the timeframe and the difficulty of implementing the improvements.

Main plan

Specifications for the main portion of the project

- Power: Battery. This allows for a higher voltage (9 V)
- Using a cable to transfer
- IMU sensor

Procedure

I like the checklist!

Phase 1:

- ☐ Research and planning

Phase 2: Initial software understanding

- ☐ Get familiar with coding on an arduino
- ☐ Analyze the communication between the microcontroller and the sensors
    - ☐ Study sensors (accuracy and calibration)
- ☐ Get wheels working

Phase 3: Parameter tuning in simulink model

Will you model the system?

- ☐ Find suitable parameters for simulation

Phase 4: Implement LQG Control

- ☐ Translate LQG from simulink to Arduino

Phase 5: Optimize real time control

- ☐ Analyze sampling time
- ☐ Check for possible errors

Phase 6: Final tests to optimize control

you'll probably need to do this from the start

- ☐ Calibration of sensors
- ☐ Tuning of parameters, as these may not be the same as the parameters found for the simulation in phase 2.

Phase 7: Further improvements (see "Further improvements" below)

Phase 8: Prepare for the presentation

Phase 9: Turn in the final report

- ☐ During the project
  - ☐ Write observations
  - ☐ Include results
- ☐ Make conclusions

Timeframe

Week	Phase								
	1	2	3	4	5	6	7	8	9
13	X								
14	X	X	X						
15-16 (break)	–	–	–	–	–	–	–	–	–
17			X	X					X
18				X	X				X
19					X	X	(X)		X
20						X	(X)		X
21						X	(X)	X	X
22 (presentation)							X	X	X

Further improvements:

1. **Bluetooth implementation**: Update the specifications to include bluetooth implementation You should definitely do this
2. **Other control theory**: Try other control theory implementations
   1. Try to implement PID
   2. Try to implement MPC
   3. Compare the various control implementations
3. **GUI**: Implementing a remote control via GUI This is required
4. **Investigate non-linear control theory:** investigate larger angles
   1. Try to implement non-linear control
   2. Compare the various control implementations
5. **Seesaw:** Implement seesaw :)